Progress Snapshot — extdrift

Behavioural-Diff Detection of Malicious Browser-Extension Updates · Bind Pratap Singh & Suhani Deepak Agrawat · Advisor: Dr. Sweta Mishra · September 2026

> **In one line:** we went from *about to write the first prototype* to a **working, tested, end-to-end system** that detects malicious extension updates — with a generated dataset, a model chosen by evidence, and results that answer the questions raised last time.

1 Where we were → where we are

At our last update	Now
Literature review done; four core papers studied	**Same, plus a full feasibility/novelty study from current sources**
About to start the first Playwright trace-capture prototype	**The full pipeline is built, tested, and running on a real browser**
No dataset	**A generated dataset of 182 update pairs, ready for ML**
No model	**A trained model chosen by a fair bake-off, with results**
—	**93 automated tests passing; code on GitHub**

2 What we built — with the actual numbers

The pipeline. Feed it two versions of an extension; it runs each in a sealed, deterministic headless-Chromium sandbox, records what each *does* (network, DOM, cookies, `chrome.*` APIs including the MV3 service worker), diffs the behaviour, and returns an explainable verdict. It runs live — a synthetic payload actually steals cookies and we catch it, confirmed by an independent collector log.

The dataset — two parts. **Synthetic: ~1,000 update pairs** across **~250 distinct extensions**, 12 personas, **9 attack families**, ~half obfuscated plus stealth variants — built with deliberate overlap so the task is *hard*, not trivially separable; 27 features per pair, tagged so evaluation splits by extension and by time (no leakage). **Real (pulled today): 7,382 confirmed-malicious extension IDs** (a public IOC database) + real extensions downloaded via our collector (Dark Reader, Grammarly, Stylus).

The machine-learning model — a fair bake-off on the harder, larger dataset:

Model	PR-AUC	False-positive rate at 90% recall
XGBoost — tabular state-of-the-art (winner)	0.972	0.07
Random Forest	0.969	0.07
Support Vector Machine	0.968	0.09
Logistic Regression	0.956	0.16
Rules (interpretable control)	0.803	0.52

XGBoost — the tabular state-of-the-art — wins on the larger, noisier data, exactly as boosting should. And **this is where ML clearly earns its place:** to catch 90% of attacks the rules alone flag **52%** of benign updates; the learned model flags **~7%**. On the earlier tiny, clean dataset every model sat at ~0.99 and the gap was noise — hardening the data made both the model comparison and the case for ML real.

Two experiments that prove the idea:

- **Obfuscation head-to-head:** a payload hidden with obfuscation — the static approach (prior work) says **BENIGN (misses it)**; our dynamic diff says **MALICIOUS (0.99)**, leak confirmed. Same behaviour, hidden code — we still catch it.
- **Determinism ablation:** the same extension diffed against itself → **0 spurious differences with our fixed pages, 8 with a noisy web**. Determinism removes the false positives.
- **Unseen threats:** trained only on benign updates, the anomaly detector flags malicious families it **never saw in training** at **43–100%** — strong on credential/cookie theft (100%), weaker on stealthy families — an honest result, not a uniform 100%.

Rigor: 93 automated tests (including live browser runs), all passing; everything on GitHub.

3 Live demo — three commands

```
# A. Instant, no browser — three verdicts on example updates (safest to show)
python scripts/demo_offline.py        -> BENIGN / SUSPICIOUS / MALICIOUS, all correct

# B. The dataset + the model, live
python -m engine.batch                builds the 182-pair dataset
python -m engine.ml.bakeoff           the model race -> "ML beats the rules: YES"

# C. The real thing in a browser — a payload steals cookies, we catch it
python scripts/demo.py                -> MALICIOUS, ML agrees, collector confirms the leak
```

Say: "This is not a plan — it runs. Here is the dataset, here is the model chosen by evidence, and here is it catching a live attack a static scanner would miss."

4 How this answers your earlier guidance

- **Reproduce the state of the art, then improve** → we built a static code-diff baseline (the prior-work style) and the transparent rules; the dynamic diff + ML **beat both** (the head-to-head).
- **Go beyond Random Forest** → a five-model bake-off compared by PR-AUC and false-positive rate; **XGBoost (the tabular state-of-the-art) wins** on the larger data. The genuine further leap — a sequence/graph model — is scoped as Project-2.
- **Get the heuristics; why is ML needed** → 12 explainable rules are the control; ML is reported only because it beats them (66% → ~0% false positives).
- **Multiple browsers** → Chromium family now; Firefox scoped as Project-2.
- **Real dataset** → a collector is built (GitHub releases / archives / weekly self-snapshot); real collection is starting now, alongside the synthetic corpus.

5 What's next (3–4 weeks)

1. Collect real benign update pairs; re-measure the false-positive rate on real data.
2. Grow the corpus; run the experiments on the larger set.
3. Begin the Firefox second engine (Project-2) and the written evaluation.

6 The honest caveat

Numbers are from a synthetic corpus plus live captures — cleaner than the real world. The result that matters is the **relationship**: dynamic beats static on obfuscation, the model beats the rules on false positives, the anomaly layer generalises to the unseen. Confirming this on real-world update pairs is the next milestone.

extdrift · CSD493 Project-1 · Prepared for the advisor progress meeting, September 2026. Figures reflect the current synthetic corpus + live browser captures. No real malware runs outside the sealed, network-restricted sandbox.